

ramanfit_swnt

June 30, 2026

1 SWNT Raman analysis

Single-wall carbon nanotube spectra, using the `ramanfit_swnt` package:

load → **Si calibration** → **RBM** (→ **diameter**, **candidate (n,m)**) → **joint D-G fit** → **2D** → **quality metrics**

Unlike the amorphous/MW-carbon D/G fit in `ramanfit.ipynb`, this:

- models the low-frequency **RBM** band (diameter fingerprint), one Lorentzian per detected sub-band, above the 180 cm^{-1} notch-filter edge,
- fits the whole **D-G region jointly over $1000\text{--}1700\text{ cm}^{-1}$** with five Lorentzians — **D**, a broad **D** disorder band ($\sim 1500\text{--}1535$), the **G /G doublet**, and **D** (~ 1600) — so the D-to-G valley is modelled instead of forced onto a straight baseline ($R^2 = 0.999$ vs 0.98 for isolated windows),
- fits the full **second-order region** (G^* , the 2D triplet, $D+D$, $2D$, $2D$) on a flat baseline,
- calibrates the shift axis against the **Si 520.7 cm^{-1}** substrate line.

```
[1]: import matplotlib.pyplot as plt
from ramanfit_swnt import (load_spectrum, analyze_swnt, load_config,
                           choose_spectrum_file)
from ramanfit_swnt.report import plot_analysis, summary_text

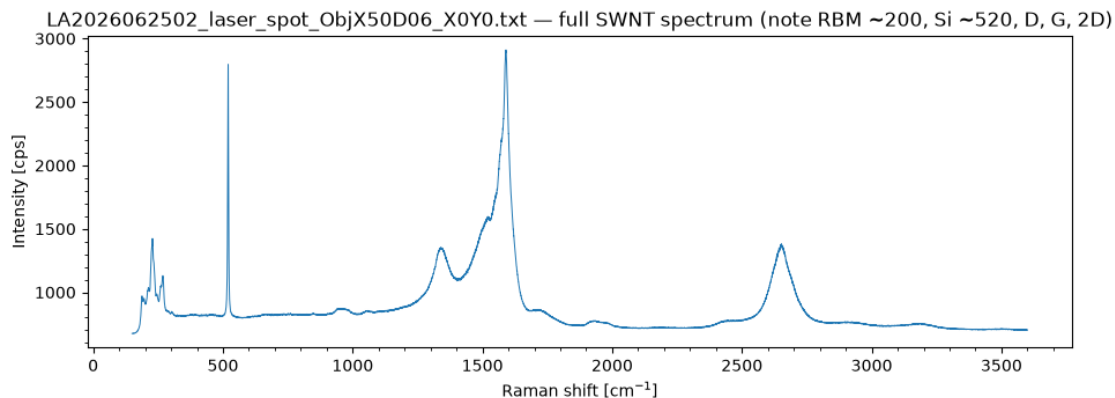
DEFAULT_INFILE = "data/LA2026062502_laser_spot_ObjX50D06_X0Y0.txt" # 532 nm
    ↪ SWNT on Si
# Pop up a file chooser; cancel (or a headless run) keeps the default.
INFILE = choose_spectrum_file(default=DEFAULT_INFILE) or DEFAULT_INFILE

x, y = load_spectrum(INFILE)
print(f"{INFILE}: {len(x)} points, {x[0]:.0f}-{x[-1]:.0f} cm-1")
```

data/LA2026062502_laser_spot_ObjX50D06_X0Y0.txt: 25732 points, $150\text{--}3600\text{ cm}^{-1}$

```
[2]: # Raw spectrum overview
import os
fig, ax = plt.subplots(figsize=(11, 3.5))
ax.plot(x, y, lw=0.7)
ax.set(xlabel="Raman shift [cm-1]", ylabel="Intensity [cps]",
       title=f"{os.path.basename(INFILE)} - full SWNT spectrum "
             f"(note RBM ~200, Si ~520, D, G, 2D)")
```

```
ax.minorticks_on()
plt.show()
```



1.1 Configure and run the pipeline

Adjust `laser_nm` (532/633/785), `gminus_lineshape` (lorentzian for semiconducting, bwf for metallic tubes) or any region in `SwntConfig`.

```
[3]: # load_config layers: SwntConfig defaults -> data/_config.json (shared) ->
# this file's RBM seed sidecar (data/<name>.rbm.json) -> kwargs below.
# So per-spectrum RBM shoulder seeds live next to the data, not in this cell.
cfg = load_config(INFILE, laser_nm=532, gminus_lineshape="lorentzian")
print("RBM extra seeds:", cfg.rbm_extra_centers or "(none - auto-detect only)")

res = analyze_swnt(x, y, cfg)
print()
print(summary_text(res))
```

RBM extra seeds: (190.0, 263.0)

Si calibration: observed 520.04 cm⁻¹, offset -0.66 cm⁻¹ (nominal 520.7)

RBM peaks (diameter, candidate (n,m)):

188.6 cm ⁻¹	d=1.31 nm	(11,9)S, (15,4)S, (12,8)S
195.9 cm ⁻¹	d=1.27 nm	-
211.2 cm ⁻¹	d=1.17 nm	(15,0)M, (10,7)M, (14,2)M
228.9 cm ⁻¹	d=1.08 nm	(8,8)M, (12,3)M, (11,5)M
269.8 cm ⁻¹	d=0.92 nm	-
263.2 cm ⁻¹	d=0.94 nm	-

D-G joint fit ($R^2=0.9956$):

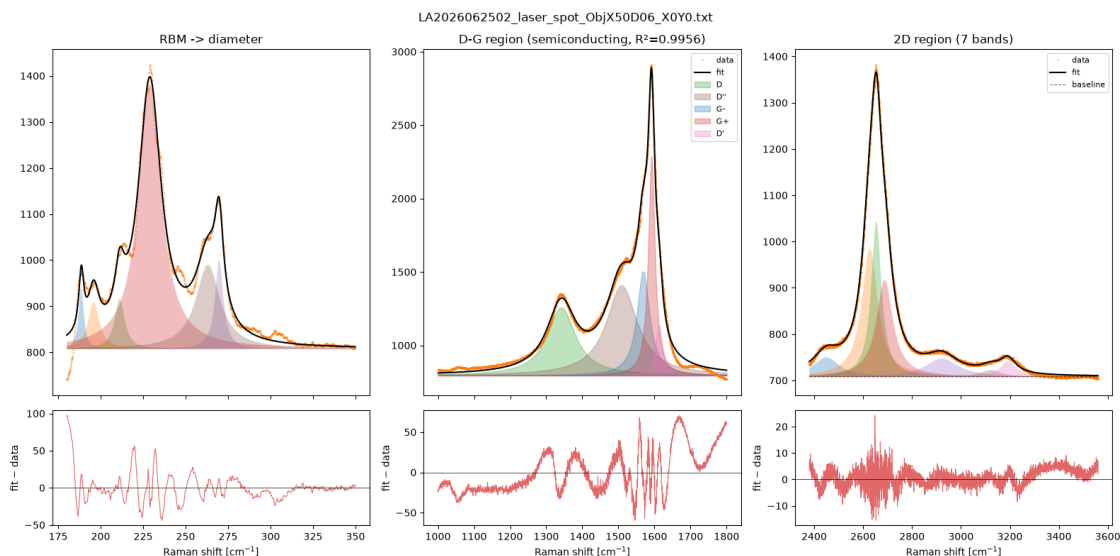
D	1341.6 cm ⁻¹ (FWHM 95)
D'	1508.8 cm ⁻¹ (FWHM 104, broad disorder band)

G+ 1591.9, G- 1568.6 (split 23.3 cm⁻¹) -> semiconducting
D' 1612.5 cm⁻¹ (FWHM 24)
I(D)/I(G+) = 0.310

2D region (7 components, R²=0.9995):

G* 2449.9 cm⁻¹ (FWHM 127, height 42)
2D 2627.7 cm⁻¹ (FWHM 79, height 277)
2D 2654.3 cm⁻¹ (FWHM 50, height 336)
2D 2688.2 cm⁻¹ (FWHM 81, height 209)
D+D' 2922.1 cm⁻¹ (FWHM 162, height 39)
2D' 3121.4 cm⁻¹ (FWHM 105, height 15)
2D' 3192.1 cm⁻¹ (FWHM 80, height 33)
dominant 2D 2654.3 cm⁻¹, I(2D)/I(G+) = 0.225

```
[4]: # Six-panel decomposition: RBM / D-G / 2D, each with data+fit over its residual
fig = plot_analysis(res, suptitle=os.path.basename(INFILE))
plt.show()
```



1.2 RBM → diameter → candidate (n,m)

(n,m) candidates come from a first-order tight-binding Kataura model and are **approximate** — see `ramanfit_swnt/kataura.py`.

```
[5]: for pk in res["rbm"]["peaks"]:
    nm = ", ".join(f"({c['n']},{c['m']}){c['type']}"
                    for c in pk.get("nm_candidates", [])[:3]) or "-"
    print(f"RBM {pk['center']:7.1f} cm⁻¹ d = {pk['diameter_nm']:.3f} nm □
    ↳{nm}")
```

RBM	188.6 cm ⁻¹	d = 1.315 nm	(11,9)S, (15,4)S, (12,8)S
RBM	195.9 cm ⁻¹	d = 1.266 nm	-
RBM	211.2 cm ⁻¹	d = 1.174 nm	(15,0)M, (10,7)M, (14,2)M
RBM	228.9 cm ⁻¹	d = 1.083 nm	(8,8)M, (12,3)M, (11,5)M
RBM	269.8 cm ⁻¹	d = 0.919 nm	-
RBM	263.2 cm ⁻¹	d = 0.942 nm	-

1.2.1 Recording RBM seeds for this spectrum

If the auto-detector missed a shoulder (a band buried on a stronger neighbour's flank), seed it manually and save it next to the data so it is applied automatically next time. Set `SAVE_RBM_SEEDS` below to the centers you want (cm⁻¹), or leave it `None` to keep the current sidecar untouched. Re-run the **Configure and run the pipeline** cell afterwards to refit with them.

```
[6]: from ramanfit_swnt import save_rbm_centers, rbm_sidecar_path

# e.g. SAVE_RBM_SEEDS = (190.0, 263.0) # set to () to clear the sidecar
SAVE_RBM_SEEDS = None

if SAVE_RBM_SEEDS is not None:
    path = save_rbm_centers(INFILE, SAVE_RBM_SEEDS)
    print(f"saved RBM seeds {SAVE_RBM_SEEDS or '(cleared)'} -> {path}")
else:
    sc = rbm_sidecar_path(INFILE)
    print(f"current sidecar: {sc if os.path.exists(sc) else '(none)'}")
```

current sidecar: data/LA2026062502_laser_spot_ObjX50D06_X0Y0.rbm.json

1.3 D-G band peaks (joint fit)

Centers, widths and uncertainties for the five components fit jointly over 1000–1700 cm⁻¹: **D**, a broad **D** disorder band filling the D-to-G valley, the **G** /**G** doublet, and **D** (~1600, defect-activated, just above G). Finite **center** ± values confirm the fit is well-determined (not railed).

```
[7]: dg = res["dg"]
print(f"D-G joint fit R² = {dg['r_squared']:.4f} ({'metallic' if
    ↪dg['metallic'] else 'semiconducting'})\n")
print(f"{'peak':4s} {'center':>9s} {'± (cm⁻¹)':>9s} {'FWHM':>7s} {'height':
    ↪>9s}")
for rec in (dg['D'], dg['Dpp'], dg['G_minus'], dg['G_plus'], dg['Dprime']):
    ce = rec.get('center_stderr')
    ce_s = f"{ce:.2f}" if ce is not None else " n/a"
    print(f"{'rec['label']':4s} {'rec['center']':9.1f} {'ce_s:>9s} "
        f"{'rec.get('fwhm', float('nan'))':7.1f} {'rec['height']':9.0f}")
```

D-G joint fit R² = 0.9956 (semiconducting)

peak	center	± (cm ⁻¹)	FWHM	height
D	1341.6	0.18	95.2	463

D''	1508.8	0.42	104.5	613
G-	1568.6	0.28	42.8	709
G+	1591.9	0.05	21.9	1491
D'	1612.5	0.25	24.2	337

1.4 2D region (multi-component)

The whole second-order region (2380–3560 cm⁻¹) is fit with a **flat (constant) baseline** at the true continuum floor — the far plateau past 2D' (~3400–3500 cm⁻¹). The apparent “hump” between the 2D and 2D' bands is not a curved background but real **D+D** (~2915) and **2D** (~3105) combination bands, so once those are fit explicitly the baseline stays horizontal. The band set matches a reference Igor multipeak fit of this sample: **G*** (~2446, iTOLA), the **2D** triplet (~2655, main + two shoulders from the diameter distribution / different (n,m) species), **D+D**, **2D**, and **2D'** (~3185, overtone of D'). The I(2D)/I(G) metric uses the dominant 2D peak.

```
[8]: t2d = res["twod_band"]
print(f"2D region: {len(t2d['peaks'])} components (R² = {t2d['r_squared']:.4f})\n")
print(f"{'band':5s} {'center':>9s} {'FWHM':>7s} {'height':>9s}")
for pk in sorted(t2d["peaks"], key=lambda r: r["center"]):
    print(f"{'pk['label']':5s} {'pk['center']':9.1f} {'pk['fwhm']':7.1f}␣
    ↪{'pk['height']':9.0f}")
print(f"\ndominant 2D {t2d['peak']['center']:.1f} cm⁻¹, I(2D)/I(G) =␣
    ↪{res['quality']['I2D_IG']:.3f}")
```

2D region: 7 components (R² = 0.9995)

band	center	FWHM	height
G*	2449.9	126.6	42
2D	2627.7	78.6	277
2D	2654.3	50.2	336
2D	2688.2	81.2	209
D+D''	2922.1	161.7	39
2D''	3121.4	104.9	15
2D'	3192.1	79.5	33

dominant 2D 2654.3 cm⁻¹, I(2D)/I(G) = 0.225

1.5 Si calibration & quality metrics

```
[9]: cal = res["calibration"]
g = res["gband"]
q = res["quality"]
print(f"Si calibration applied: {cal['applied']} (offset {cal['offset']:+.2f}␣
    ↪cm⁻¹)")
print(f"G = {g['G_plus']['center']:.1f}, G = {g['G_minus']['center']:.1f}␣
    ↪cm⁻¹)")
print(f"character: {'metallic' if g['metallic'] else 'semiconducting'})
```

```

quality = "low defects" if q['ID_IG'] < 0.2 else ("moderate defects" if
↪q['ID_IG'] < 0.5 else "high defects")
print(f"I(D)/I(G) = {q['ID_IG']:.3f}    ({quality})")
print(f"2D = {q['twod_center']:.1f} cm-1, I(2D)/I(G) = {q['I2D_IG']:.3f}")

```

Si calibration applied: True (offset -0.66 cm⁻¹)
 G = 1591.9, G = 1568.6 cm⁻¹
 character: semiconducting
 I(D)/I(G) = 0.310 (moderate defects)
 2D = 2654.3 cm⁻¹, I(2D)/I(G) = 0.225

1.6 Save a summary

```

[10]: with open("swnt_summary.txt", "w") as f:
        f.write(summary_text(res))
    print("wrote swnt_summary.txt")

```

wrote swnt_summary.txt

1.7 Export this notebook to PDF

Running the cell below renders the executed notebook to `ramanfit_swnt.pdf` (needs `pandoc` + `xelatex`; it falls back to HTML if those are missing). It is skipped automatically during a non-interactive `nbconvert` run so it cannot recurse into itself.

```

[11]: import os
        # Import from the submodule directly: a kernel that imported ramanfit_swnt
        # before export_pdf was added still resolves this (no kernel restart needed).
        from ramanfit_swnt.export import export_pdf

        NOTEBOOK = "ramanfit_swnt.ipynb"
        if os.environ.get("RAMANFIT_NO_DIALOG"):
            print("skipping PDF export (non-interactive nbconvert run)")
        else:
            export_pdf(NOTEBOOK)

```

skipping PDF export (non-interactive nbconvert run)